

第7章 网络流算法

- 7.1 最大流问题
 - 网络流及其性质
 - Ford-Fulkerson算法
 - Dinic有效算法
- 7.2 最小费用流
 - Floyd算法
 - 最小费用流的负回路算法
 - 最小费用流的最短路径算法
- 7.3/4 网络流的应用

7.1 最大流问题

网络流及其性质

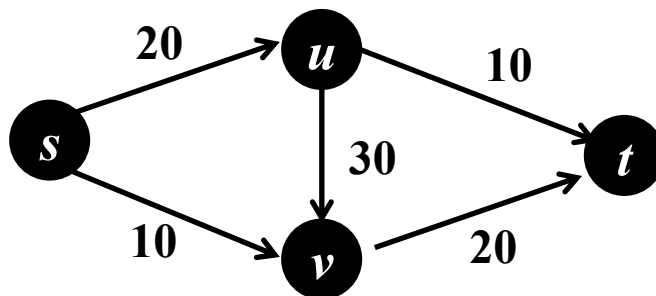
容量网络 $N = \langle V, E, c, s, t \rangle$

其中 $N = \langle V, E \rangle$ 是有向连通图, 记 $n = |V|$, $m = |E|$,

$c: E \rightarrow R^*$ 是边的容量,

s 和 t 是两个特殊的顶点, s 称作发点(源), t 称作收点(汇),

其余顶点称作中间点.



最大流的定义

设 $f: E \rightarrow R^*$ 满足下述条件:

(1) 容量限制 $\forall \langle i, j \rangle \in E, f(i, j) \leq c(i, j),$

(2) 平衡条件 $\forall i \in V - \{s, t\},$

$$\sum_{\langle j, i \rangle \in E} f(j, i) = \sum_{\langle i, j \rangle \in E} f(i, j)$$

称 f 是 N 上的一个可行流, 称 s 的净流出量 $v(f)$ 为 f 的流量, 即

$$v(f) = \sum_{\langle s, j \rangle \in E} f(s, j) - \sum_{\langle j, s \rangle \in E} f(j, s)$$

流量最大的可行流称作最大流.

最大流问题

最大流问题 求给定容量网络上的最大流.

$$\max v(f)$$

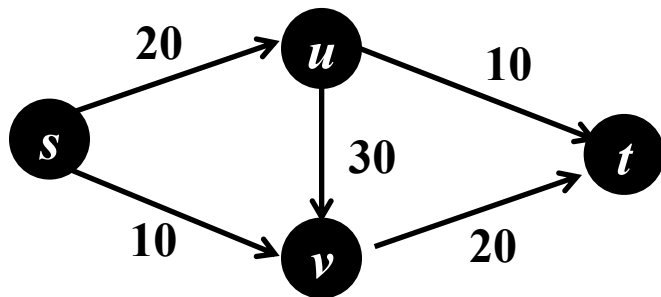
$$\text{s.t. } f(i,j) \leq c(i,j), \quad \langle i,j \rangle \in E$$

$$\sum_{\langle j,i \rangle \in E} f(j,i) - \sum_{\langle i,j \rangle \in E} f(i,j) = 0, \quad i \in V - \{s,t\}$$

$$v(f) - \sum_{\langle s,j \rangle \in E} f(s,j) + \sum_{\langle j,s \rangle \in E} f(j,s) = 0$$

$$f(i,j) \geq 0, \quad \langle i,j \rangle \in E$$

$$v(f) \geq 0$$



最大流

$$f(s,u)=20, f(u,t)=10, f(s,v)=10,$$

$$f(u,v)=10, f(v,t)=20$$

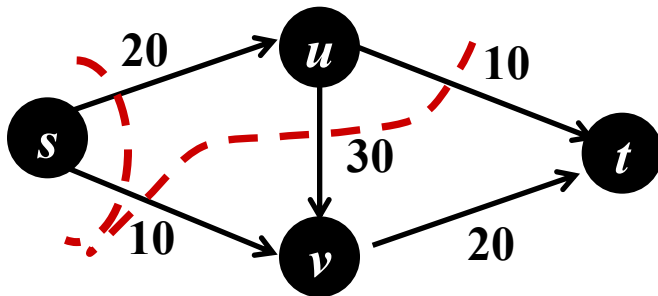
最小割

定义 设容量网络 $N=\langle V,E,c,s,t\rangle$, $A\subset V$ 且 $s\in A$, $t\in V-A$, 称
 $(A,V-A)=\{\langle i,j\rangle \mid \langle i,j\rangle\in E \text{ 且 } i\in A, j\in V-A\}$

为 N 的**割集**,

$$c(A,V-A) = \sum_{\langle i,j\rangle\in(A,\bar{A})} c(i,j)$$

为割集 $c(A,V-A)$ 的**容量**; 容量最小的割集称作**最小割集**.



$$A=\{s,u\}$$

$$c(\{s,u\},\{v,t\})=50$$

$$A=\{s\}$$

$$c(\{s\},\{u,v,t\})=30$$

引理

引理1 设容量网络 $N=\langle V, E, c, s, t \rangle$, f 是 N 上的任一可行流, $A \subset V$ 且 $s \in A, t \in V-A$, 则 $v(f) = f(A, V-A) - f(V-A, A)$

证

$$\begin{aligned} v(f) &= \sum_{\langle s, j \rangle \in E} f(s, j) - \sum_{\langle j, s \rangle \in E} f(j, s) \\ &= \sum_{\langle s, j \rangle \in E} f(s, j) - \sum_{\langle j, s \rangle \in E} f(j, s) + \sum_{i \in A - \{s\}} \left\{ \sum_{\langle i, j \rangle \in E} f(i, j) - \sum_{\langle j, i \rangle \in E} f(j, i) \right\} \\ &= \sum_{i \in A} \left\{ \sum_{\langle i, j \rangle \in E} f(i, j) - \sum_{\langle j, i \rangle \in E} f(j, i) \right\} \\ &= \sum_{i \in A} \sum_{\langle i, j \rangle \in E} f(i, j) - \sum_{i \in A} \sum_{\langle j, i \rangle \in E} f(j, i) \\ &= \sum_{\substack{i \in A \\ j \in A}} \sum_{\langle i, j \rangle \in E} f(i, j) + \sum_{\substack{i \in A \\ j \in V-A}} \sum_{\langle i, j \rangle \in E} f(i, j) - \sum_{\substack{i \in A \\ j \in A}} \sum_{\langle j, i \rangle \in E} f(j, i) - \sum_{\substack{i \in A \\ j \in V-A}} \sum_{\langle j, i \rangle \in E} f(j, i) \\ &= \sum_{\langle i, j \rangle \in (A, V-A)} f(i, j) - \sum_{\langle j, i \rangle \in (V-A, A)} f(j, i) \end{aligned}$$

引理

引理2 设 f 是任一可行流, $(A, V-A)$ 是任一割集, 则

$$v(f) \leq c(A, V-A)$$

引理3 设 f 是一个可行流, $(A, V-A)$ 是一个割集. 如果

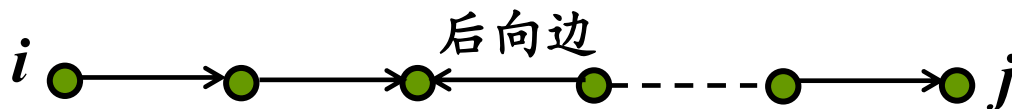
$$v(f) = c(A, V-A)$$

则 f 是最大流, $(A, V-A)$ 是最小割集.

i - j 增广链

定义 设容量网络 $N=\langle V,E,c,s,t\rangle$, f 是 N 上的一个可行流.

- (1) N 中流量等于容量的边称作**饱和边**, 流量小于容量的边称作**非饱和边**.
- (2) 流量等于0的边称作**零流边**, 流量大于0的边称作**非零流边**.
- (3) 不考虑边的方向, N 中从顶点 i 到 j 的一条边不重复的路径称作 **i - j 链**. i - j 链的方向是从 i 到 j . 链中与链的方向一致的边称作**前向边**, 与链的方向相反的边称作**后向边**.
- (4) 如果链中所有前向边都是非饱和的, 所有后向边都是非零流的, 则称这条链为 **i - j 增广链**.

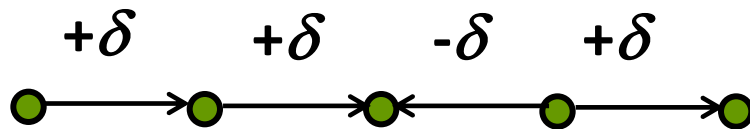


最大流的性质

定理1 可行流 f 是最大流 $\Leftrightarrow$ 不存在关于 f 的 s - t 增广链.

证 必要性. 假设 P 是一条可行流 f 的 s - t 增广链, 修改量 δ 等于 P 上所有前向边容量与流量之差及所有后向边流量的最小值. 令

$$f'(i, j) = \begin{cases} f(i, j) + \delta & \langle i, j \rangle \text{ 是 } P \text{ 的前向边} \\ f(i, j) - \delta & \langle i, j \rangle \text{ 是 } P \text{ 的后向边} \\ f(i, j) & \text{否则} \end{cases}$$



f' 是可行流, 且 $v(f') = v(f) + \delta$, 与 f 是最大流矛盾.

最大流的性质（续）

充分性. 假设不存在关于 f 的 s - t 增广链. 令

$$A = \{j \in V \mid \text{存在关于 } f \text{ 的 } s\text{-}j \text{ 增广链}\}$$

$s \in A, t \notin A$. $\forall \langle i, j \rangle \in (A, V-A)$, 必有 $f(i, j) = c(i, j)$. 否则, 可以把 s - i 增广链通过非饱和的 $\langle i, j \rangle$ 延伸到 j , 与 $j \notin A$ 矛盾.

$\forall \langle j, i \rangle \in (V-A, A)$, 必有 $f(j, i) = 0$. 否则, 可以把 s - i 增广链通过非零流的 $\langle j, i \rangle$ (后向边) 延伸到 j , 与 $j \notin A$ 矛盾. 于是, 由引理1

$$v(f) = \sum_{\langle i, j \rangle \in (A, \bar{A})} f(i, j) - \sum_{\langle j, i \rangle \in (\bar{A}, A)} f(j, i) = c(A, \bar{A})$$

又由引理3, 得证 f 是最大流.

定理2 (最大流最小割集定理)

容量网络的最大流的流量等于最小割集的容量.

Ford-Fulkerson算法

设计思想

从给定初始可行流 (通常取零流) 开始
取当前可行流的 s - t 增广链 P , 修改 P 上流量得到新的可行流.
重复进行, 直到不存在 s - t 增广链为止.

标号法

从 s 开始给顶点作标号, 直到 t 得到标号为止.
顶点 j 得到标号表示已找到从 s 到 j 的增广链.

标号为 (l_j, δ_j)

δ_j 等于 s - j 链上可增加流量的最大值

$l_j = +i$ 表示链是从 i 到 j 的且 $\langle i, j \rangle$ 是前向边

$l_j = -i$ 表示链是从 i 到 j 的且 $\langle j, i \rangle$ 是后向边

顶点状态 已标号已检查的, 已标号未检查的, 未标号的.

FF算法伪码

Ford-Fulkerson算法

1. $f \leftarrow 0$ // 零流为初始可行流
2. $T \leftarrow \{s\}, R \leftarrow V - \{s\}, l_s \leftarrow \Delta, \delta_s \leftarrow +\infty$ // T :已标未查, R :未标,
3. while $T \neq \emptyset$ do
4. 取 $i \in T, T \leftarrow T - \{i\}$ // 检查顶点 i
5. for 所有 R 中与 i 邻接的 j do
6. if $\langle i, j \rangle \in E$ 且 $f(i, j) < c(i, j)$ then
7. $l_j \leftarrow +i, \delta_j \leftarrow \min\{\delta_i, c(i, j) - f(i, j)\}, R \leftarrow R - \{j\}, T \leftarrow T \cup \{j\}$
8. if $j = t$ then goto 13 // 找到 s - t 增广链
9. if $\langle j, i \rangle \in E$ 且 $f(j, i) > 0$ then
10. $l_j \leftarrow -i, \delta_j \leftarrow \min\{\delta_i, f(j, i)\}, R \leftarrow R - \{j\}, T \leftarrow T \cup \{j\}$
11. if $j = t$ then goto 13 // 找到 s - t 增广链
12. return f // 无 s - t 增广链, 结束

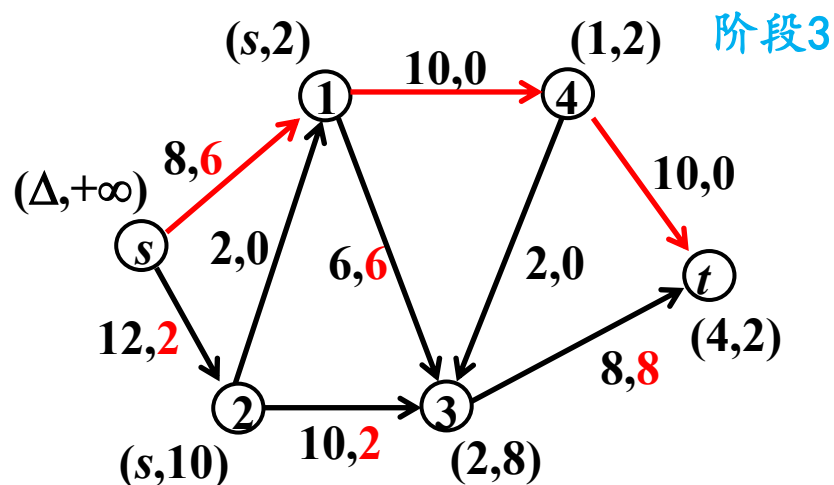
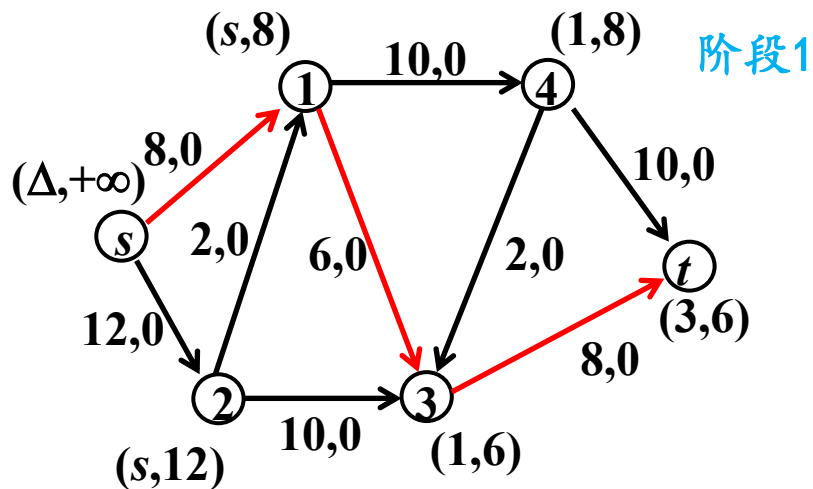
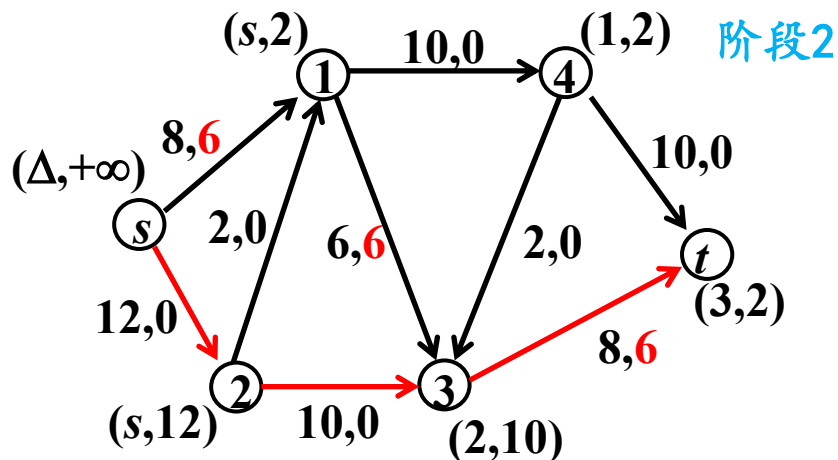
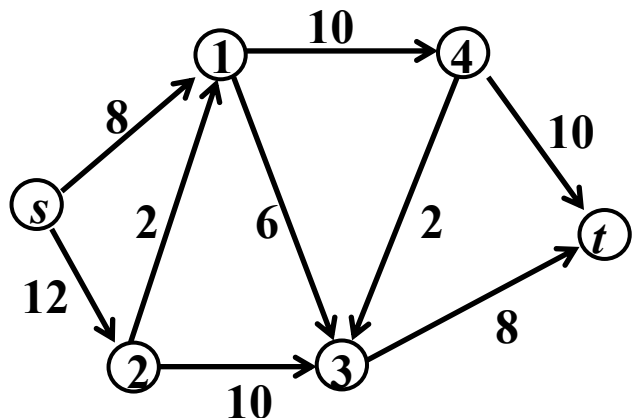
FF算法伪码（续）

13. $\delta \leftarrow \delta_t$ // 沿增广链回溯, 修改流量
14. if $l_j = \Delta$ then goto 2 // 重新开始下一阶段标号
15. if $l_j = +i$ then
16. $f(i, j) \leftarrow f(i, j) + \delta, j \leftarrow i$
17. if $l_j = -i$ then
18. $f(j, i) \leftarrow f(j, i) - \delta, j \leftarrow i$
19. goto 14

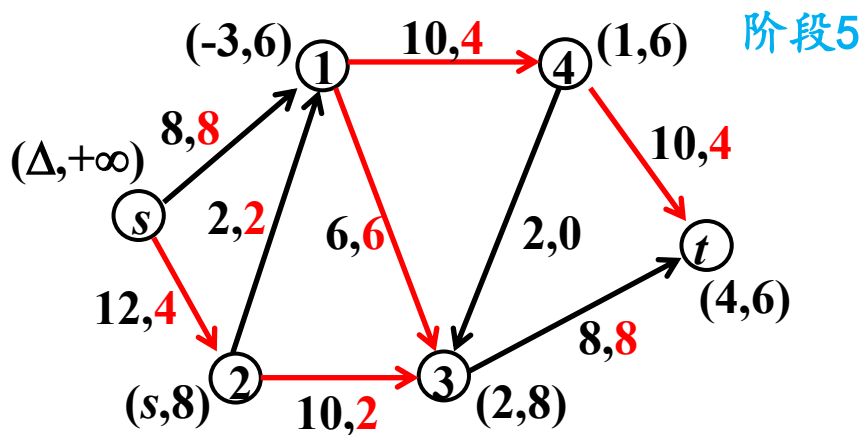
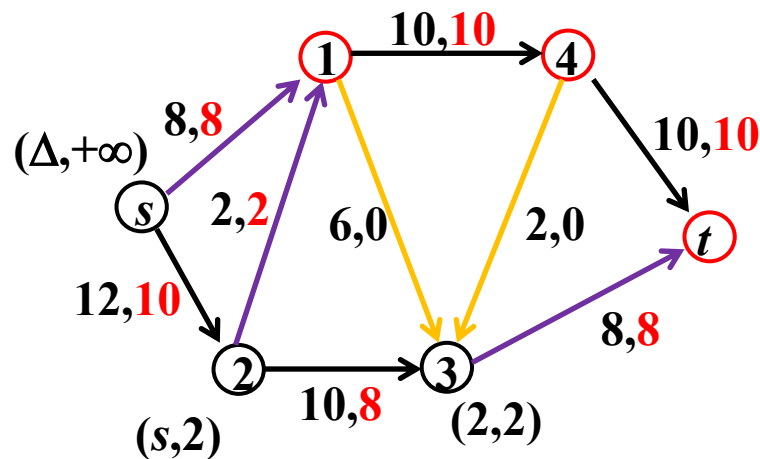
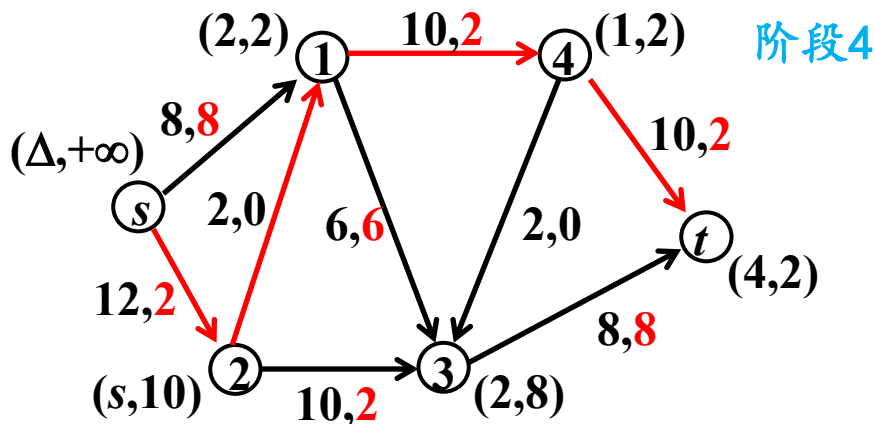
每个阶段的工作:

- 通过标记过程寻找一条 $s-t$ 增广链
- 沿链回溯修改链上的流值

算法运行实例：例1



算法运行实例：例1（续）



解： $v(f)=18$

$f(s,1)=8, f(s,2)=10, f(1,3)=0,$
 $f(1,4)=10, f(2,1)=2, f(2,3)=8,$
 $f(3,t)=8, f(4,3)=0, f(4,t)=10,$

最小割 $(\{s,2,3\}, \{1,4,t\})$

$c(\{s,2,3\}, \{1,4,t\})=18$

算法运行时间

时间复杂度

假设所有容量都是正整数, 则算法时间复杂度为 $O(mC)$, 其中

$$C = \sum_{\langle s, j \rangle \in E} c(s, j)$$

最大流量 $v^* \leq C$. δ 是正整数, 每次流量至少加1, 至多 C 个阶段. 而每阶段标号和修改增广链流量需要 $O(m)$.

提高效率的途径

每次求最短的 s - t 增广链

一次标号修改尽可能多条 s - t 增广链上的流量

辅助网络

定义 设容量网络 $N = \langle V, E, c, s, t \rangle$, f 是 N 上的一个可行流.

关于 f 的**辅助网络** $N(f) = \langle V, E(f), ac, s, t \rangle$, 其中

$$E^+(f) = \{ \langle i, j \rangle \mid \langle i, j \rangle \in E \text{ 且 } f(i, j) < c(i, j) \}$$

$$E^-(f) = \{ \langle j, i \rangle \mid \langle i, j \rangle \in E \text{ 且 } f(i, j) > 0 \}$$

$$E(f) = E^+(f) \cup E^-(f)$$

$$ac(i, j) = \begin{cases} c(i, j) - f(i, j), & \langle i, j \rangle \in E^+(f) \\ f(j, i), & \langle i, j \rangle \in E^-(f) \end{cases}$$

ac 称作**辅助容量**. $N(f)$ 也是容量网络.

7.2 最小费用流

定义

- (1) 在容量网络 $N=\langle V, E, c, s, t \rangle$ 中添加单位费用 $w: E \rightarrow R^*$, 称作容量-费用网络, 记作 $N=\langle V, E, c, w, s, t \rangle$.
- (2) 设 f 是 N 上的一个可行流, 称

$$w(f) = \sum_{\langle i, j \rangle \in E} w(i, j) f(i, j)$$

为 f 的费用.

- (3) 所有流量为 v_0 的可行流中费用最小的称作流量 v_0 的最小费用流.

最小费用流问题

给定容量-费用网络 N 和流量 v_0 , 求流量 v_0 的最小费用流.

容量-费用网络的辅助网络

设容量-费用网络 $N = \langle V, E, c, w, s, t \rangle$, 关于可行流 f 的辅助网络 $N(f) = \langle V, E(f), ac, aw, s, t \rangle$, 其中辅助费用

$$aw(i, j) = \begin{cases} w(i, j), & \text{若 } \langle i, j \rangle \in E^+(f) \\ -w(j, i), & \text{若 } \langle i, j \rangle \in E^-(f) \end{cases}$$

引理9 设 f 是容量-费用网络 N 上的可行流, g 是辅助网络 $N(f)$ 上的可行流, $f' = f + g$, 则

$$w(f') = w(f) + aw(g)$$

圈流及其性质

定义 设 C 是 N 中边不重复的回路, C 上的**圈流** h^C 定义如下:

- (1) $\forall \langle i, j \rangle \in E(C), h^C(i, j) = \delta;$
- (2) $\forall \langle i, j \rangle \in E - E(C), h^C(i, j) = 0,$

其中 $\delta > 0$ 是 h^C 的 **环流量**.

性质

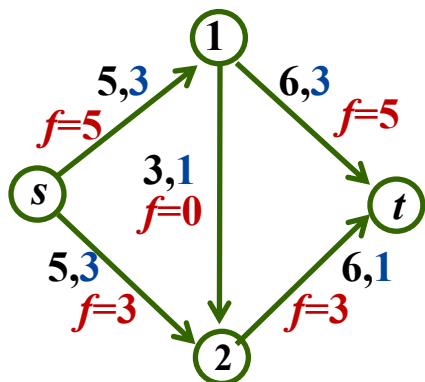
- (1) h^C 为可行流, 且 $v(h^C) = 0, w(h^C) = \delta \cdot w(C)$
- (2) 设 f 是 N 上的一个可行流, h^C 是 $N(f)$ 上的一个圈流,
 $f' = f + h^C$ 是 N 上的可行流, 且

$$v(f') = v(f), \quad w(f') = w(f) + \delta \cdot w(C)$$

如果 $w(C) < 0$, 则 $w(f') < w(f)$.

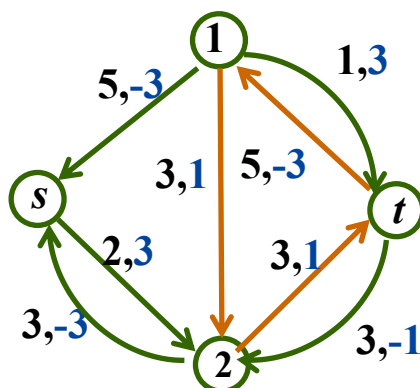
实例

例4 容量费用网络 N , 可行流 f , 容量、费用、流值如下:



N 与 f

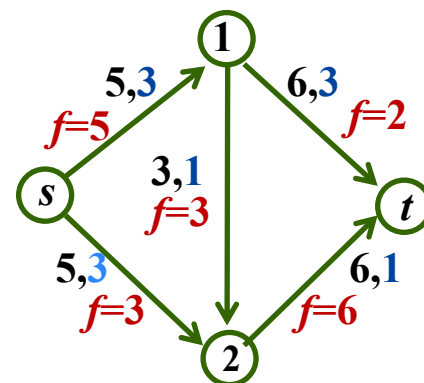
$$v_0=8, w(f)=42.$$



$N(f)$ 与 C

$$aw(C) = -1, \delta = 3$$

$$aw(h^C) = -3$$



$$f_1 = f + h^C$$

$$w(f_1) = 42 - 3 = 39$$

最小费用流的负回路算法

问题 容量-费用网络 $N=\langle V, E, c, w, s, t \rangle$, 求 N 中流量为 v_0 的最小费用流.

负回路算法的设计思想

1. 首先求一个流量 v_0 的可行流 f
2. 如果 $N(f)$ 中存在权 aw 的负回路 C , 令 $f' \leftarrow f + h^C$
3. 重复进行, 直至辅助网络中不存在权 aw 的负回路为止.

可证明: 当 $N(f)$ 中不存在权 aw 的负回路时, f 是最小费用流.

最短路径与负回路

求带负权的最短路径和检测负回路的算法

负回路 赋权有向图 $D=\langle V,E,w\rangle$ 中权为负数的回路

命题1 赋权有向图 D 中任意两点之间都有最短路径或不存在路径当且仅当 D 中不含负回路.

证 假设 D 中有负回路 C , i 是 C 上的一个顶点, 那么从 i 到 j 的路径可以先重复走 C 若干次再到 j , 因而 i 到 j 的路径的权可以任意小.

假设 D 中不存在负回路 C . 对任意两点 i 和 j , 如果从 i 到 j 的路径中有两个相同顶点, 删去它们之间的路径(回路)仍是从 i 到 j 的路径, 其权不增加. 只需考虑从 i 到 j 顶点都不相同的路径. 而从 i 到 j 顶点都不相同的路径只有有限条, 故一定存在最短路径.

最短路径的Floyd算法

动态规划方法:

$d^{(k)}(i, j)$: 从 i 到 j 经过号码不大于 k 的最短路径的长度.

递推方程

$$d^{(0)}(i, j) = w(i, j), \quad 1 \leq i, j \leq n$$

$$d^{(k)}(i, j) = \min\{d^{(k-1)}(i, j), d^{(k-1)}(i, k) + d^{(k-1)}(k, j)\},$$
$$1 \leq i, j \leq n \text{ 且 } i, j \neq k, 1 \leq k \leq n$$

规定: $\forall i \in V, w(i, i) = 0; \quad \forall \langle i, j \rangle \notin E, w(i, j) = +\infty.$

当 D 中不存在负回路时, i 到 j 的距离 $d(i, j) = d^{(n)}(i, j).$

当 D 中存在负回路时, 设负回路 C 经过 i , 除 i 外顶点的最大号码是 k , 则必有 $d^{(k)}(i, i) < 0.$

Floyd算法

$h^{(k)}(i, j)$: 从 i 到 j 经过号码不大于 k 的最短路径中 i 的下一顶点

递推关系

$$h^{(0)}(i, j) = \begin{cases} j & \langle i, j \rangle \in E \\ 0 & \text{else} \end{cases}$$

$$h^{(k)}(i, j) = \begin{cases} h^{(k-1)}(i, j) & d^{(k-1)}(i, j) \leq d^{(k-1)}(i, k) + d^{(k-1)}(k, j) \\ h^{(k-1)}(i, k) & \text{else} \end{cases}$$

$$1 \leq i, j \leq n ; i, j \neq k, 1 \leq k \leq n$$

算法：利用递推公式迭代计算 $d^{(k)}(i, j)$ 和 $h^{(k)}(i, j)$,

$i, j, k = 1, 2, \dots, n$

$d^{(n)}(i, j)$ 是 i 到 j 的最短路长度

$h^{(n)}(i, j)$ 用于追踪从 i 到 j 的路径

算法伪码

算法4 最小费用流的负回路算法

1. 调用最大流算法, 若求得流量 v_0 的可行流 f , 则转2; 若最大流量小于 v_0 , 则输出“无流量 v_0 的可行流”, 计算结束.
2. 构造 $N(f)$.
3. 用Floyd算法检测 $N(f)$ 中是否存在权 aw 的负回路. 若存在负回路 C , 则转4; 若不存在负回路, 则输出 f , 计算结束.
4. 令 $\delta \leftarrow \min\{ ac(i, j) \mid \langle i, j \rangle \in E(C) \}$ // h^C 的环流量为 δ
5. $\forall \langle i, j \rangle \in E(C)$, 若 $\langle i, j \rangle \in E$, 令 $f(i, j) \leftarrow f(i, j) + \delta$;
若 $\langle j, i \rangle \in E$, 令 $f(j, i) \leftarrow f(j, i) - \delta$. // $f \leftarrow f + h^C$
6. 转2

其他算法

最短路径算法

设计思想：

1. 从一个初始的最小费用流 f (如零流) 开始,
2. 如果 $v(f) < v_0$, 找一条费用最少的 s - t 增广链 P , 修改 P 上的流量, 得到新的可行流 f' .
3. 重复进行, 直至流量等于 v_0 为止.

若存在负回路, 用 Floyd 算法计算最短路径;

若不存在负回路, 可以用 Ford 算法.

小结

- 容量网络的定义
- 最大流与最小割
- 最大流最小割定理
- Ford-Forkson 最大流算法
- 最小费用流
- 负回路最小费用流算法